

CREDITCOIN · ATTESTCOIN · SEPTEMBER 2026

StateLift

Bounded budget recovery.
Safe payment handoff.

One payment goal. At most one compliant payment.
Recover the budget at D. Let dedicated B carry the late obligation.

Technical whitepaper

Real testnet prototype · Experimental economics

Contents

1. The operator's problem
 2. Identities and commitments
 3. Source-chain at-most-once enforcement
 4. Creditcoin accounting and deadline semantics
 5. Attestcoin is on the funds path
 6. Four observable paths
 7. Underwriting economics
 8. Limits and what remains unproven
- Evidence and references

Technical whitepaper · testnet prototype · September 2026

A payment operator can recover a reserved budget at an agreed clearing deadline and hand the same payment goal to another executor. If an old payment is proven late, that round's dedicated, fully funded guarantee pays the executor. The recipient receives at most one compliant payment for the goal.

All monetary values and deadlines in the demonstration are experimental parameters. This paper does not claim commercial insurance pricing, customer adoption or production readiness.

1. The operator's problem

Consider a Creditcoin payment operator with a fixed USDC obligation to an Ethereum supplier. An executor agrees to pay from its Ethereum inventory in exchange for a CTC reward. The supplier does not need to adopt Creditcoin.

The difficult case is an executor disappearing while the payment's outcome is unknown on Creditcoin. Waiting indefinitely ties up the operator's budget. Refunding without allocating the old obligation leaves someone bearing an unpriced loss. Starting a new independent payment can pay the supplier twice.

StateLift targets this specific operational failure. It is not a universal cross-chain platform, a prediction of credit quality, or a guarantee that every participant can exit during every outage. Its purchased result combines three rights: a clearing deadline for the operator's R, replacement execution under the same G, and funded compensation for a late old payment.

2. Identities and commitments

A payment goal G binds the Creditcoin chain and escrow, its owner, a business reference, source token, recipient and exact amount. Terms that change during a retry are deliberately excluded. Both chains derive the same G independently.

Each execution round E has an immutable signed quote: round number, source payment deadline T , Creditcoin clearing deadline D , principal/reward R , dedicated guarantee $B \geq R$, premium π , guarantor identity and quote expiry/nonce. A new round has fresh $R/B/\pi$ but retains G .

The guarantee is funded before acceptance. Capital backing one round cannot back another accepted round simultaneously. A guarantor with insufficient available capital cannot open more exposure merely by signing a quote.

3. Source-chain at-most-once enforcement

GoalRouter accepts USDC ReceiveWithAuthorization. The authorization's recipient is the router; its nonce binds G , round, winner and T . The router checks the payment deadline, receives exactly the expected tokens, forwards them to G 's recipient and writes a permanent fill record atomically.

All compliant rounds collide on the same fills[G] entry. Changing the executor, signature nonce or round does not permit another compliant payment. Failed transactions may still incur gas; at-most-once refers to the USDC payment, not free execution or prevention of unrelated gifts to the recipient.

This rule avoids a common retry error: a per-attempt authorization nonce prevents replay of one attempt but does not prevent a new attempt from paying the same business obligation again.

4. Creditcoin accounting and deadline semantics

The escrow separates four balance buckets:

contract balance = held principal R + available guarantee capital + locked per-round B + withdrawable credits.

Premium π is charged at round acceptance and credited to the guarantor. It is not counted as guarantee backing. The winner's reward is R ; the prototype does not add a separate bonus on top of R . Credits represent ownership inside the escrow and are withdrawn by the credited account.

If a valid payment fact is accepted for settlement before D , the executor receives R from the operator's principal and B unlocks. At or after D , the operator retains or recovers R and the executor receives R from that round's guarantee; any excess $B - R$ becomes available again.

An operator can claim R at D without supplying a payment proof. This does not declare that the payment never happened. B remains locked until a usable winning or expiry fact resolves the round. A late settlement cannot take R back from the operator by winning a transaction-order race: allocation follows the contract's clearing-deadline rule.

A winner fact for one round proves that other rounds of the same G cannot win. It releases their B and any unreturned R . Alternatively, a proof that the router remains unfilled at a source timestamp at or after T retires that round while keeping G open.

5. Attestcoin is on the funds path

The deployed source is Ethereum Sepolia, chain ID 11155111. The deployed destination is Creditcoin testnet, chain ID 102031. Attestcoin source chain key 1 refers to Sepolia in this environment; it is not an EVM chain ID.

The integration uses the official prover and native verifier at 0x00FD2. A real HeaderAnchor transaction records the canonical hash of a preceding payment block. The official proof authenticates that anchor's transaction and receipt. RootInbox checks success, emitter, source domain, height and hash, then validates the supplied full block header against that hash.

The payment block's receipts are fetched from the source node and its Merkle-Patricia receipt trie is reconstructed. The reconstructed root must match the authenticated header. GoalFillFactVerifier verifies the GoalRouter event and extracts the goal, round, winner, recipient, amount, T and source timestamp. StateLiftGoalEscrow consumes this verified fact in settleRound to allocate CTC.

The RPC, proof builder and submitting process transport evidence; they are not authorized to insert an arbitrary root or assign a winner. An authenticated transaction existing is not sufficient: status, emitter and business conditions are checked by application contracts.

The observed prover requires a 32-source-block reorg-protection window plus attestation availability. Proof generation and destination inclusion add further latency. These observations are not a latency SLA. The source anchor must execute within the EVM's 256-block BLOCKHASH history window; permissionless recovery still depends on data and timely anchoring availability.

6. Four observable paths

Path	What actually happens	Allocation required
Normal	Executor pays before T; proof settles before D	R to winner, B released
Late proof	Executor pays before T; operator claims at D; proof arrives later	Operator keeps R; that round's B pays winner
Unpaid handoff	Old executor never pays; R1 returned at D; executor 2 pays under G	R2 to executor 2; winning fact releases old B1; one supplier payment

The local suite executes all four paths, failed second payments, wrong facts, deadlines, negative storage facts and balance invariants. Tests: full-suite baseline 107/107; updated negative-fact suite 13/13, including two expiries then a third-round settlement. The recorded local demonstration completed 49 CLI commands and 13 invariant snapshots. Local USDC and Attestcoin substitutes are explicitly labelled and are not evidence of real testnet integration.

Real testnet receipts, official responses and per-role before/after snapshots are stored separately in evidence/testnet. Roles use distinct addresses controlled by one test harness; this establishes mechanics, not independent economic counterparties. A deployment or proof-validity result alone does not establish settlement: the relevant flow must include confirmed settle and changed credits.

All four real testnet paths have confirmed Creditcoin state-changing transactions. Normal settlement is `0xeffe0ce3...`, block 5,481,035. Late settlement is `0x742f0927...`, block 5,481,040. Round-2 handoff settlement is `0xa532ba0a...`, block 5,481,066, followed by old-round guarantee release at block 5,481,067. The unfilled expiry path released B at `0xc281cefe...`, block 5,481,512, while leaving G open.

A separately mined duplicate payment transaction reverted on Sepolia; both payer and recipient USDC balances remained unchanged. After the normal, late and relay paths, before the later expiry run, each credited role withdrew to its own wallet; gas-adjusted native balance changes were checked and all escrow accounting buckets reconciled to zero (evidence/testnet/withdrawals.json). The complete testnet execution audit, including historical CTC balance-bucket changes at settlement blocks, is maintained in evidence/testnet/audit.json. This is stronger evidence than a proof-builder response or a deployment address.

7. Underwriting economics

Let q be the probability that a compliant paid round ultimately consumes B because settlement is late; τ its actual guarantee lock duration; c annual capital cost; g guarantor-paid gas/proof costs; and o operating costs. A simple zero-recovery lower bound is:

$$\pi_{\min} = qR + BcE[\tau]/365 \text{ days} + E[g] + o + \text{risk margin}.$$

The demo uses $\pi/R=0.6\%$. Even with zero other costs, a late-loss probability at or above 0.6% exhausts that premium. On-chain gas must be measured and allocated to the actor that pays it, including transactions subsidized by the demonstration harness.

The measured normal settlement consumed 1,971,381 Creditcoin gas (0.0009856905 tCTC); root acceptance added 0.0001242045 tCTC. Those two transactions alone cost 0.001109895 tCTC, above the demo premium of 0.0006 tCTC. The demo deployer paid these costs, so they are a visible subsidy, not demonstrated underwriting profit. Source-chain gas is denominated separately in test ETH. Full per-sender costs are in evidence/testnet/gas-costs.json.

The real testnet example uses $R=B=0.1$ tCTC and 0.1 test USDC. Those numbers conserve faucet funds; they are not an exchange rate or an insurance quote. Short late-case deadlines deliberately trigger coverage and must not be extrapolated into sustainable pricing.

For N simultaneous exposures, locked capital must cover the sum of their dedicated guarantees. A shared prover outage can make many rounds late together. With 100 hypothetical rounds of $R=B=1000$ and $\pi=6$, total premium is 600, while one late loss costs 1000 and a fully correlated event can cost 100000, before costs. Average independent-event assumptions miss that tail.

There is no finite bound on B's lock time when usable source facts remain unavailable. Bounded budget recovery protects the operator's principal; it does not create riskless capital recycling for guarantors.

8. Limits and what remains unproven

The prototype has no evidence of paying customers, market-clearing quotes, independent guarantors or profitable underwriting. CTC/USDC exchange risk, executor inventory needs, USDC blacklisting/upgrades, chain reorganizations and source/destination outages remain relevant. CTC

chain unavailability can prevent wall-clock withdrawal at D; gas and premium are not refunded and R's purchasing power is not protected.

No claim is made that the code has received a professional security audit. Negative facts depend on an authenticated state root, the correct router code hash, and a source timestamp beyond T. Missing evidence is never treated as evidence that payment did not occur.

Production work would require measured latency/loss distributions, correlated exposure caps, independent capital and execution providers, key-management review, adversarial audit, source anchoring operations and commercial validation.

Evidence and references

- README.md: product entry and reproducible local commands.
- evidence/full-recheck.log: 107/107 baseline; multi-round-validation.log: updated negative-fact suite, 13/13.
- evidence/three-acts.md and three-acts-transitions.json: local demonstration, explicitly mock-labelled.
- evidence/testnet/deployment.json: six production deployments and runtime code hashes.
- evidence/testnet/flow-`{normal,late,relay,expiry}`.json: transactions and per-role accounting.
- docs/TESTNET-INTEGRATION.md: official endpoints, proof pipeline and resumption commands.
- research/underwriting-model.md: pricing assumptions and correlated-loss examples.